



Mock Exam

May 29, 2026

First and Last name: _____

Student ID: _____

Signature: _____

General Remarks

- Please check that you have all 22 pages of this exam.
- There are 120 points in total, and the exam is 120 minutes. **Don't spend too much time on a single question!** The maximum of points is not required for the best grade!
- Remove all material from your desk that is not permitted by the examination regulations, including electronic devices, which should be in airplane mode.
- Write your answers directly on the exam sheets. If you need more space, make sure you put your name and your **student-ID**-number on top of each supplementary sheet.
- Immediately inform an assistant in case you are not able to take the exam under regular conditions. Later complaints are not accepted.
- Attempts to cheat/defraud lead to immediate exclusion from the exam and can have judicial consequences.
- Please use a black or blue pen to answer the questions.
- Provide only one solution to each exercise. Cancel invalid solutions clearly.
- Once the time is up, **immediately** drop the pen and stop writing. Do not wait for an additional warning as there will be none.

Topic	Max Points	Achieved
1 — MDPs, Bellman Equations & Dynamic Programming	25	
2 — Model-Free Learning & Function Approximation	20	
3 — Policy Gradients, Actor-Critic & PPO	35	
4 — LLM Post-Training	40	
Total	120	

Grade:

Topic 1 — MDPs, Bellman Equations & Dynamic Programming (25 pts)

Setup: Coffee, Code, or Sleep

Lina is an MSc student in Basel. Each evening starts with her feeling either tired, focused, or wired. Depending on how she feels, she chooses between drinking a coffee or resting before getting to work. Her choice affects both how much research progress she makes that evening and how she'll feel the next evening.

- When **tired**: coffee gives progress 1 and leaves her focused; resting gives progress 0 and she stays tired.
- When **focused**: coffee gives progress 3 and leaves her wired; resting gives progress 2 and she ends up tired.
- When **wired**: coffee makes her crash hard — progress -3 and she's tired again the next evening; resting gives progress 4 and she's focused again.

Lina's fixed evening routine: tired $\rightarrow$ always coffee; focused $\rightarrow$ fair coin flip; wired $\rightarrow$ always rest.

Lina is patient but not infinitely so: if her supervisor offers her either 1 unit of progress today or 4 units of progress two evenings from now, she is indifferent between the two.

Q1.1 — Model the MDP [6 pts]

Formalize Lina's evenings as an MDP. Give:

(a) the state space $\mathcal{S}$

1 pts

.....

(b) the action space $\mathcal{A}$

1 pts

.....

(c) the transition and reward function as a table

2 pts

.....

.....

.....

(d) Lina's policy $\pi(a \mid s)$ for every state

1 pts ☐

.....

.....

(e) the discount factor γ

1 pts ☐

.....

.....

Q1.2 — Bellman expectation equations (4 points)

Write the Bellman expectation equations for $v_\pi(T), v_\pi(F), v_\pi(W)$ under π . You do **not** need to solve the system.

4 pts ☐

.....

.....

.....

.....

Q1.3 — Compute $q_\pi(s, a)$ (5 points)

For the remainder of the topic, you may use:

$$v_\pi(T) = 3.5, \quad v_\pi(F) = 5, \quad v_\pi(W) = 6.5.$$

Compute the action-value function $q_\pi(s, a)$ for all six state–action pairs.

5 pts ☐

.....

.....

.....

.....

.....

.....

Q1.4 — Code: value iteration (4 points)

The following snippet implements synchronous **value iteration** on Lina's MDP. It uses two helper functions, `reward(s, a)` and `next_state(s, a)`, which implement the dynamics from Q1.1. Fill in the two missing lines: (i) the Bellman optimality backup, and (ii) a convergence check.

4 pts

```
1 import numpy as np
2 gamma = 0.5
3 n_states, n_actions = 3, 2
4
5 V = np.zeros(n_states)
6 for _ in range(1000):
7     V_new = np.zeros(n_states)
8     for s in range(n_states):
9         q_values = []
10        for a in range(n_actions):
11            r = reward(s, a)
12            sp = next_state(s, a)
13            q_values.append( _____ )    # (i) fill in
14        V_new[s] = max(q_values)
15    if _____ :                        # (ii) fill
16        in
17        break
18    V = V_new
19 print(V)
```

Q1.5 — Multiple Choice Questions (4 points)

Choose one answer only. +1 for a correct answer, no negative points.

4 pts

- (a) The Bellman expectation equation for v_π differs from the Bellman optimality equation for v_* in that:
- ☐ A) The expectation equation has a max over actions; the optimality equation has an average.

- ☐ B) The expectation equation averages over actions under π ; the optimality equation takes a max over actions.
- ☐ C) The expectation equation uses γ ; the optimality equation does not.
- ☐ D) They are identical for deterministic policies.

(b) For a finite MDP with discount $\gamma \in [0, 1)$, the Bellman expectation operator T^π is:

- ☐ A) A γ -contraction in the ℓ_∞ norm.
- ☐ B) A γ -contraction only when π is deterministic.
- ☐ C) Non-expansive but not a contraction.
- ☐ D) A contraction only when rewards are bounded by 1.

(c) Policy iteration on a finite MDP with discount $\gamma \in [0, 1)$ is guaranteed to:

- ☐ A) Diverge if rewards are negative.
- ☐ B) Converge to v_* in a finite number of iterations.
- ☐ C) Converge only asymptotically (never finitely).
- ☐ D) Require $\gamma = 1$ to converge.

(d) The “policy improvement step” in policy iteration sets the new policy π' to:

- ☐ A) $\pi'(s) \in \arg \max_a q_\pi(s, a)$ using the current policy's q_π .
- ☐ B) $\pi'(s) \in \arg \max_a q_{\pi'}(s, a)$ using the new policy's q .
- ☐ C) The same as π but with random tie-breaking.
- ☐ D) The policy that uniformly mixes actions.

Q1.6 — One-step greedy improvement (2 points)

Using your $q_\pi(s, a)$ values from Q1.3, write down the one-step greedy policy $\pi'(s) \in \arg \max_a q_\pi(s, a)$ for each state, and say in one sentence whether Lina should change her routine.

2 pts ☐

.....

.....

.....

.....

.....

.....

Topic 2 — Model-Free Learning & Function Approximation (20 pts)

Q2.1 — Monte Carlo vs. TD(0) targets (5 points)

Consider one observed episode of three steps in some MDP, with discount factor $\gamma = \frac{1}{2}$:

$$s_0 \xrightarrow{r=2} s_1 \xrightarrow{r=4} s_2 \xrightarrow{r=-1} \text{end}.$$

Before this episode, the current value estimates are

$$\hat{V}(s_0) = 1, \quad \hat{V}(s_1) = 3, \quad \hat{V}(s_2) = 0.$$

- (a) Compute the **Monte Carlo** return G_0 (the target for $\hat{V}(s_0)$ under MC).

2 pts

.....
.....

- (b) Compute the **TD(0)** target for $\hat{V}(s_0)$ (using one step into s_1).

2 pts

.....
.....

- (c) In one sentence, state which of the two targets has higher *variance*, which has higher *bias*, and why.

1 pts

.....
.....

Q2.2 — SARSA vs. Q-learning on the same transition (6 points)

Sam runs a small grocery shop in Basel. Every day he chooses whether to **restock** (K) or **wait** (W), and the shop's state moves between **Empty**, **Stocked**, and **Overstocked**. He does not know the dynamics in advance — he learns by trial and error.

Sam uses an ϵ -greedy behavior policy with learning rate $\alpha = 0.1$ and discount $\gamma = 0.9$. At some point during training, his Q-table is:

	K	W
E	5	2
S	8	14
O	1	4

Sam observes the transition $(s, a, r, s') = (E, K, 3, S)$. His ϵ -greedy policy then *actually picks* $a' = K$ as the next action in state S .

- (a) Compute the updated value of $Q(E, K)$ under **SARSA**.

2 pts

.....

.....

.....

- (b) Compute the updated value of $Q(E, K)$ under **Q-learning**.

2 pts

.....

.....

.....

- (c) In one sentence, explain why the two updates differ even though both methods observed the *same* transition.

2 pts

.....

.....

.....

Q2.3 — Code: linear semi-gradient TD(0) (5 points)

The following Python class implements linear function approximation with semi-gradient TD(0). The value estimate is $\hat{v}(s, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s)$ where $\mathbf{x}(s)$ is the feature vector for state s . **Two lines** of the update method are missing — fill them in.

```

1 import numpy as np
2
3 class LinearTD:
4     def __init__(self, feature_fn, alpha=0.01, gamma=1.0):
5         self.feature_fn = feature_fn
6         self.alpha      = alpha
7         self.gamma      = gamma
8         self.w          = np.zeros(feature_fn.dim)
9
10    def value(self, state):
11        return np.dot(self.w, self.feature_fn.features(state))
12
13    def update(self, state, reward, next_state, done):
14        x_s      = self.feature_fn.features(state)
15        v_s      = ----- # (i) current
16                   prediction
17        v_next = 0.0 if done else self.value(next_state)
18        delta  = reward + self.gamma * v_next - v_s
19        self.w = ----- # (ii) semi-
20                   gradient update
21        return delta

```

(a) Fill in line (i): the current value prediction $\hat{v}(s, \mathbf{w})$.

1.5 pts

(b) Fill in line (ii): the semi-gradient TD update.

2.5 pts

(c) One sentence: why is this called the *semi*-gradient update rather than a full gradient?

1 pts

.....

.....

.....

Q2.4 — Multiple Choice Questions (4 points)

Choose one answer only. +1 for a correct answer, no negative points.

4 pts

(a) The “deadly triad” in RL is the combination of:

- ☐ A) Replay buffers, target networks, and the use of GPU acceleration.

- ☐ B) Function approximation, ϵ -greedy exploration, and stochastic dynamics.
- ☐ C) Deep neural networks, large discrete action spaces, and sparse rewards.
- ☐ D) Function approximation, bootstrapping, and off-policy learning.

(b) In DQN, the role of the *target network* Q_{θ^-} is best described as:

- ☐ A) Speeding up gradient computation by caching the loss across the previous mini-batches.
- ☐ B) Replacing ϵ -greedy exploration with a more principled Boltzmann sampling rule over Q-values.
- ☐ C) Stabilizing the bootstrapped TD target so that the regression target does not shift with every gradient step.
- ☐ D) Computing exact Bellman-optimal values whenever the dynamics of the MDP are known to the agent.

(c) Compared to Monte Carlo, the TD(0) update is:

- ☐ A) Unbiased and has higher variance because each return is the full Monte Carlo sum.
- ☐ B) Biased due to bootstrapping but has lower variance than Monte Carlo.
- ☐ C) Unbiased and has lower variance, since bootstrapping never introduces any bias at all.
- ☐ D) Biased due to bootstrapping and also has strictly higher variance than Monte Carlo.

(d) Which of the following is the correct **trajectory importance-sampled return** for off-policy evaluation, where π is the target policy and b is the behavior policy?

- ☐ A) $G_t^{\text{corr}} = G_t \cdot \prod_{k=t}^{T-1} \frac{b(A_k | S_k)}{\pi(A_k | S_k)}$
- ☐ B) $G_t^{\text{corr}} = G_t \cdot \sum_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}$
- ☐ C) $G_t^{\text{corr}} = G_t \cdot \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}$
- ☐ D) $G_t^{\text{corr}} = G_t + \prod_{k=t}^{T-1} \frac{\pi(A_k | S_k)}{b(A_k | S_k)}$

Topic 3 — Policy Gradients, Actor–Critic & PPO (35 pts)

Q3.1 — The score function trick (6 points)

The policy gradient theorem relies on the **score function (log-derivative) trick**:

$$\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}}[f(x)] = \mathbb{E}_{x \sim p_{\theta}}[f(x) \nabla_{\theta} \log p_{\theta}(x)].$$

- (a) Derive this identity, starting from $\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}}[f(x)] = \nabla_{\theta} \int p_{\theta}(x) f(x) dx$. State clearly where you use $\nabla_{\theta} \log p_{\theta}(x) = \nabla_{\theta} p_{\theta}(x) / p_{\theta}(x)$.

4 pts

.....
.....
.....
.....

- (b) In the RL setting, x corresponds to an action a , $p_{\theta}(x)$ to the policy $\pi_{\theta}(a | s)$, and $f(x)$ to the return $R(\tau)$ produced by the environment. In one sentence, explain why this trick is necessary in this fashion — i.e. why we cannot simply backpropagate through the environment.

2 pts

.....
.....

Q3.2 — What REINFORCE actually does to the policy (8 points)

In REINFORCE, after an episode the policy parameters are updated using $\nabla_{\theta} \log \pi_{\theta}(a | s)$ scaled by the episode return.

- (a) An action a was taken in state s , and the episode return was **positive and large**. In which direction does the update move the probability $\pi_{\theta}(a | s)$ — up or down — and why? Answer in terms of the role of $\nabla_{\theta} \log \pi_{\theta}(a | s)$ and the sign of the return.

3 pts

.....

.....

.....

.....

- (b) Now suppose the return was **negative**. What happens to $\pi_\theta(a \mid s)$, and what does this mean intuitively for that action?

2 pts

.....

.....

.....

.....

- (c) In the *same* state s , two episodes took two different actions a_1 and a_2 , with returns $G = 10$ and $G = 12$ respectively. Explain why situations like this could be wasteful for learning. Then suppose we know a state-dependent baseline $b(s) = 11$. Explain what would change about the two updates if we subtract $b(s)$ to the returns.

3 pts

.....

.....

.....

.....

.....

Q3.3 — What the critic buys you (8 points)

An actor-critic agent updates its actor with the advantage estimate $\hat{A}_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$.

- (a) Suppose the critic is untrained and outputs $V_\psi(s) \approx 0$ for all states. What does $\hat{A}_t$ reduce to, and which classical policy-gradient algorithm does the actor's update then become?

3 pts

.....

.....

.....

- (b) As the critic is trained and becomes accurate, the *variance* of the actor's learning signal drops sharply. But a new source of error appears. What is it, and where does it come from?

3 pts ☐

.....

.....

.....

- (c) In one sentence, state the bias–variance trade-off between plain REINFORCE and actor–critic.

2 pts ☐

.....

.....

.....

Q3.4 — Why PPO clips (6 points)

PPO maximizes the clipped surrogate objective

$$L(\theta) = \mathbb{E} \left[\min \left(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t \right) \right], \quad \rho_t = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

- (a) In words, what does the ratio ρ_t measure? What does $\rho_t > 1$ mean about how the new policy treats the sampled action a_t compared to the old policy?

2 pts ☐

.....

.....

.....

- (b) Consider a transition with a **positive** advantage, $A_t > 0$. Plain policy-gradient ascent would push ρ_t as large as possible. Explain what the min/clip does here, and *why* letting ρ_t grow

without bound would be harmful.

2 pts

☐

.....

.....

.....

- (c) Still with $A_t > 0$, suppose the ratio has already grown past $1 + \epsilon$. By evaluating the two arguments of the min, show that the surrogate's gradient w.r.t. θ becomes **zero** in this region, and state the practical effect on the update.

2 pts

☐

.....

.....

.....

.....

.....

Q3.5 — Multiple Choice Questions (6 points)

Choose one answer only. +1 for a correct answer, no negative points.

6 pts

☐

- (a) Which expression is the **policy gradient theorem** (trajectory-return form)?

- ☐ A) $\nabla_{\theta} J = \mathbb{E}[\sum_t \nabla_{\theta} \pi_{\theta}(a_t | s_t) G_t]$
- ☐ B) $\nabla_{\theta} J = \mathbb{E}[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t]$
- ☐ C) $\nabla_{\theta} J = \mathbb{E}[\sum_t \log \pi_{\theta}(a_t | s_t) \nabla_{\theta} G_t]$
- ☐ D) $\nabla_{\theta} J = \mathbb{E}[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) + G_t]$

- (b) Which expression is the **one-step TD error** used as a critic's advantage estimate?

- ☐ A) $\delta_t = r_t + \gamma V_{\psi}(s_t) - V_{\psi}(s_{t+1})$
- ☐ B) $\delta_t = r_{t+1} + \gamma V_{\psi}(s_{t+2}) - V_{\psi}(s_{t+1})$
- ☐ C) $\delta_t = r_t + \gamma \max_a Q(s_t, a) - V_{\psi}(s_t)$

☐ D) $\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$

(c) Subtracting a baseline $b(s_t)$ in REINFORCE keeps the gradient **unbiased** provided that:

- ☐ A) $b(s_t)$ depends only on the state, not on the action taken at t .
- ☐ B) $b(s_t)$ is exactly equal to the constant zero for every state.
- ☐ C) $b(s_t)$ equals the realised return G_t on every single timestep.
- ☐ D) $b(s_t)$ depends on both the action and the next state visited.

(d) The $\text{GAE}(\lambda)$ estimator is defined as

$$\hat{A}_t^{\text{GAE}(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t).$$

What does $\text{GAE}(\lambda)$ recover when $\lambda = 1$?

- ☐ A) The one-step TD residual
- ☐ B) The full trajectory advantage $G_t - V_\phi(s_t)$
- ☐ C) The Monte Carlo return G_t
- ☐ D) The policy gradient without any baseline

(e) In the actor-critic loss, an entropy bonus $-c \mathcal{H}[\pi_\theta(\cdot \mid s_t)]$ is often added. Its primary purpose is to:

- ☐ A) Encourage continued exploration by penalising premature determinism.
- ☐ B) Make the critic's value estimates converge faster to true returns.
- ☐ C) Guarantee the policy-gradient estimator becomes exactly unbiased.
- ☐ D) Replace the need for a discount factor in long-horizon tasks.

(f) The **trust-region / KL idea** behind TRPO and PPO is best summarised as:

- ☐ A) Force the critic to exactly equal the Monte Carlo return at every state.
- ☐ B) Restrict each policy update so the new policy stays close to the old one.
- ☐ C) Replace the discount factor γ with a learned, state-dependent value.
- ☐ D) Sample actions uniformly at random to guarantee sufficient exploration.

Topic 4 — LLM Post-Training

(40 pts)

Q4.1 — Supervised Fine-Tuning (6 points)

- (a) For a prompt x with a demonstration response $y^* = (y_1^*, \dots, y_T^*)$, write the SFT training objective (the quantity maximized over θ).

2 pts

.....
.....

- (b) In one sentence: why does SFT produce the *reference* policy π_{ref} used by later stages, rather than the final aligned policy?

2 pts

.....
.....

- (c) SFT is equivalent to minimizing which divergence, and between which two distributions?

2 pts

.....
.....

Q4.2 — Bradley–Terry & DPO (14 points)

Setup. For a single prompt x , a preference dataset contains pairs $(y^+ \succ y^-)$. The Bradley–Terry model for a reward function r is

$$P(y^+ \succ y^- \mid x) = \sigma(r(x, y^+) - r(x, y^-)), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

DPO does not train an explicit reward model; it uses the *implicit reward*

$$r(x, y) = \beta(\log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x)) + C(x),$$

where $C(x)$ depends only on the prompt. Write $\Delta_\theta = \log \pi_\theta(y^+ \mid x) - \log \pi_\theta(y^- \mid x)$ and $\Delta_{\text{ref}} = \log \pi_{\text{ref}}(y^+ \mid x) - \log \pi_{\text{ref}}(y^- \mid x)$.

- (a) Substitute the implicit reward into the Bradley–Terry model and show that

$$P(y^+ \succ y^- \mid x) = \sigma(\beta(\Delta_\theta - \Delta_{\text{ref}})).$$

State explicitly at which step the prompt-only term $C(x)$ disappears, and why.

5 pts

.....

.....

.....

.....

.....

- (b) Hence write the per-pair DPO training loss (the negative log-likelihood of the observed preference $y^+ \succ y^-$).

2 pts

.....

.....

- (c) Using the loss you derived, take $\beta = 0.5$ and $\Delta_{\text{ref}} = 0$ for two preference pairs. Pair I: $\log \pi_{\theta}(y^+) = -0.2$, $\log \pi_{\theta}(y^-) = -3.0$. Pair II: $\log \pi_{\theta}(y^+) = -1.0$, $\log \pi_{\theta}(y^-) = -1.2$. Compute the DPO logit $u_{\theta} = \beta(\Delta_{\theta} - \Delta_{\text{ref}})$ for each pair. Both pairs are already ranked correctly — state which pair produces the *larger* gradient update, and explain why in one line.

6 pts

.....

.....

.....

.....

.....

.....

- (d) DPO can be derived from the KL-regularized RLHF objective, but removes two components that RLHF requires. Name both.

1 pts

.....

.....

Q4.3 — Group Relative Policy Optimization (12 points)

Setup. For a prompt x , GRPO samples a group of G completions and scores them, obtaining rewards $r^{(1)}, \dots, r^{(G)}$. Instead of a learned value network, GRPO uses the sampled group itself as the baseline. Two prompts are each given a group of $G = 4$ completions, with rewards

Prompt 1: $(5, 5, 5, 5)$, Prompt 2: $(0, 0, 0, 4)$.

- (a) Write the GRPO group-relative advantage $A^{(i)}$ in terms of the group rewards $r^{(1)}, \dots, r^{(G)}$ (mean-centred and normalised by the group standard deviation, with the convention that the advantage is set to zero when the group standard deviation is zero). Then state what $A^{(i)}$ becomes when all G rewards are equal — as for **Prompt 1** — and what this implies GRPO learns from such a group.

4 pts

.....
.....
.....

- (b) Using your definition from (a), compute the advantage vector for **Prompt 2**.

2 pts

.....
.....
.....

- (c) A colleague rescales and shifts every reward by the same affine map, $r \mapsto 10r + 3$, and re-runs GRPO on Prompt 2. Do the advantages change? Justify in one line.

2 pts

.....
.....

- (d) In one sentence: why does GRPO need no learned value (critic) network, unlike PPO?

1 pts

-
-
- (e) While monitoring your GRPO training, you notice that, after a bunch of steps, the KL divergence term has increased significantly. Discuss what does this imply and why this could happen. What mitigation would you try in your next run?

3 pts

.....

.....

.....

.....

Q4.4 — Multiple Choice Questions (8 points)

Choose one answer only. +1 for a correct answer, no negative points.

8 pts

- (a) The SFT checkpoint is used in later stages as:
- ☐ A) the reward model that scores candidate responses.
 - ☐ B) the reference policy π_{ref} the aligned policy is kept close to.
 - ☐ C) the value network used to compute the advantages.
 - ☐ D) the dataset of human preference comparisons.
- (b) While monitoring your DPO training, you notice that the average response length has steadily increased. Which of the following is a likely cause?
- ☐ A) The KL penalty grows linearly with response length, forcing the policy to produce shorter responses over time.
 - ☐ B) The reference policy assigns equal log-probability to responses of all lengths, creating no length preference.
 - ☐ C) The implicit reward favours longer responses, since increasing the log-probability of a longer sequence yields a larger absolute change in the reward signal.
 - ☐ D) The reward model was trained on a dataset biased towards short responses, causing the policy to overcompensate.

- (c) The KL-regularized optimum is $\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$. If $\pi_{\text{ref}}(y | x) = 0$ for some response, then $\pi^*(y | x)$ is:
- ☐ A) zero for every finite $\beta > 0$, no matter how large the reward.
 - ☐ B) positive provided the reward $r(x, y)$ is large enough.
 - ☐ C) equal to $1/Z(x)$, independent of the reference.
 - ☐ D) undefined, because of a division by zero.
- (d) As $\beta \rightarrow \infty$ in $\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$, the optimal policy:
- ☐ A) concentrates entirely on the highest-reward response.
 - ☐ B) approaches the reference policy π_{ref} .
 - ☐ C) becomes the uniform distribution over responses.
 - ☐ D) becomes completely independent of the reference.
- (e) Compared to PPO, DPO has a fundamental data requirement:
- ☐ A) DPO requires a separate reward model to be retrained after every gradient step.
 - ☐ B) DPO requires the preferred and rejected responses to come from different prompts.
 - ☐ C) DPO requires responses to be generated by the reference policy at training time.
 - ☐ D) DPO requires a static, pre-collected preference dataset.
- (f) In RLHF the per-token reward is the reward-model score *minus* a KL-to-reference penalty. Removing the KL penalty and optimizing the score alone typically causes:
- ☐ A) faster convergence to genuinely higher-quality text.
 - ☐ B) reward hacking: the proxy score rises while true quality degrades.
 - ☐ C) the policy to collapse exactly onto the reference policy.
 - ☐ D) the reward model to become perfectly calibrated.
- (g) You are training a model on a mathematical reasoning task where the only available signal is whether the final answer is correct or not. Which algorithm is most directly applicable?
- ☐ A) SFT, because binary correctness signals can be converted to demonstration data by keeping only correct responses.
 - ☐ B) DPO, because correct and incorrect responses form natural preference pairs.
 - ☐ C) PPO, because it requires a learned reward model trained on correctness labels.
 - ☐ D) GRPO, because it is designed for verifier-style rewards and requires no learned value network.
- (h) PPO, DPO, and GRPO can be viewed as:
- ☐ A) KL-regularized post-training objectives, with different data and estimators.

- ☐ B) entirely unrelated objectives with nothing in common.
- ☐ C) supervised next-token prediction methods with no reward signal.
- ☐ D) exact dynamic programming algorithms on the token-level MDP.

Supplementary Sheet

Supplementary Sheet